Lab 01 – Identifying Number Formats

IPv4, IPv6 & MAC Address Identification, Binary Conversion, and Subnet Analysis

Name: Chase Cantrell
Course/Section: IS-3413
Date: 2/3/2025

INTRODUCTION

This lab report documents the identification and analysis of network addressing components on a physical host machine, including IPv4, IPv6, MAC, subnet mask, and default gateway addresses. Each address type is examined in its native notation and converted to its binary equivalent using standard octet-by-octet decomposition. The lab concludes with an applied subnet mask analysis — both standard and inverted — to derive the network and host address portions from a given IP address, and an extended exercise using a Python-generated random IPv4 address.

Breakpoint 1 — Network Configuration Retrieval via ipconfig

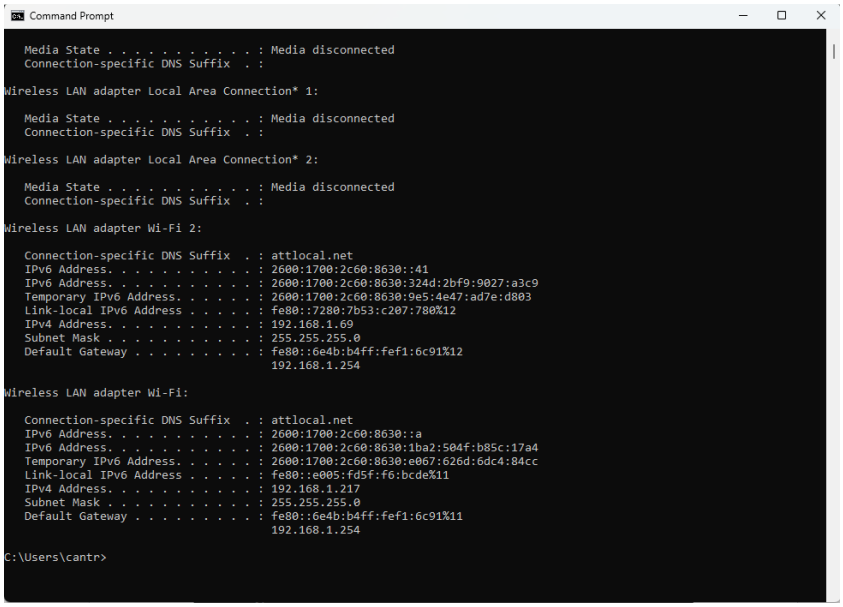


Figure 1: Output of the ipconfig command displaying full network adapter configuration

The Windows ipconfig command was executed from the Command Prompt to retrieve the network configuration details for the host machine. The output displayed configuration data across multiple network adapters. The relevant values for the active wireless adapter (Wi-Fi 2) were recorded and are defined below:

IPv4 Address: `192.168.1.69` — *32-bit numerical address that uniquely identifies a device on a local network*

IPv6 Address: `2600:1700:2c60:8630::41` — *128-bit address used to identify devices on the modern internet*

Subnet Mask: `255.255.255.0` — *Distinguishes the network portion from the host portion of an IP address*

Default Gateway: `fe80::6ea4:b4ff:fe1f:6c91%12 / 192.168.1.254` — *IP address of the router serving as the network's exit point*

MAC Address: `CC-28-AA-65-B5-E3` — *48-bit hardware identifier permanently assigned to the network interface card*

Breakpoint 2 — Binary Conversion of IPv4 and IPv6 Addresses

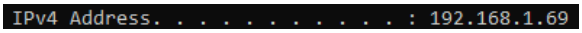


Figure 2: IPv4 address highlighted in ipconfig output

Each octet of the IPv4 address was converted to its 8-bit binary representation using successive division by 2, recording remainders in reverse order:

`192` → `11000000`
`168` → `10101000`
`1` → `00000001`
`69` → `01000101`

Final IPv4 binary representation:

```
11000000.10101000.00000001.01000101
```

Figure 3: IPv6 address highlighted in ipconfig output

Each 16-bit hexadecimal group of the IPv6 address was converted to its binary equivalent:

`2600` → `0010011000000000`
`1700` → `0001011100000000`
`2c60` → `0010110001100000`
`8630` → `1000011000110000`
`41` → `0000000001000001`

Final IPv6 binary representation (compressed groups represented by '::'):

```
0010011000000000:0001011100000000:0010110001100000:1000011000110000::00000000
01000001
```

Breakpoint 3 — Binary Conversion of MAC Address

Figure 4: MAC address highlighted in ipconfig output

Each hexadecimal byte of the MAC address was independently converted to its 8-bit binary equivalent:

CC → 11001100
28 → 00101000
AA → 10101010
65 → 01100101
B5 → 10110101
E3 → 11100011

Final MAC address binary representation:

```
11001100-00101000-10101010-01100101-10110101-11100011
```

Breakpoint 4 — Subnet Mask and Host Mask Analysis

A bitwise AND operation between the IP address and the subnet mask isolates the network portion of the address, yielding the network address. Inverting the subnet mask and performing the same operation isolates the host portion.

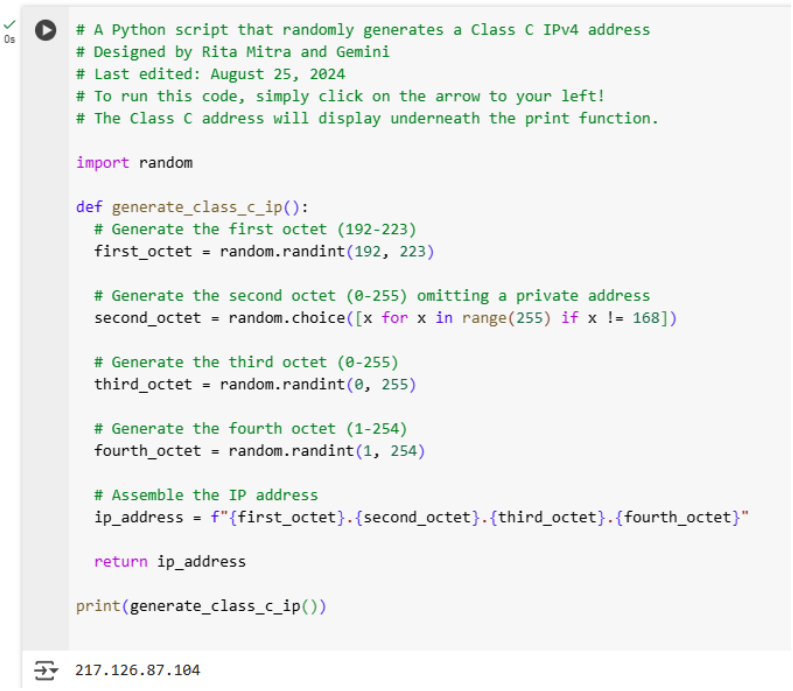
Network Address (Standard AND):

IP Address	11000000.10101000.00000001.01000101 (192.168.1.69)
Subnet Mask	11111111.11111111.11111111.00000000 (255.255.255.0)
Network Addr	11000000.10101000.00000001.00000000 → 192.168.1.0

Host Address (Inverted Subnet Mask AND):

IP Address	11000000.10101000.00000001.01000101 (192.168.1.69)
Inv. Subnet	00000000.11111111.11111111.11111111 (0.255.255.255)
Host Addr	00000000.10101000.00000001.01000101 → 0.168.1.69

Breakpoint 5 — Random IPv4 Address via Python Script



```python
# A Python script that randomly generates a Class C IPv4 address
# Designed by Rita Mitra and Gemini
# Last edited: August 25, 2024
# To run this code, simply click on the arrow to your left!
# The Class C address will display underneath the print function.

import random

def generate_class_c_ip():
  # Generate the first octet (192-223)
  first_octet = random.randint(192, 223)

  # Generate the second octet (0-255) omitting a private address
  second_octet = random.choice([x for x in range(255) if x != 168])

  # Generate the third octet (0-255)
  third_octet = random.randint(0, 255)

  # Generate the fourth octet (1-254)
  fourth_octet = random.randint(1, 254)

  # Assemble the IP address
  ip_address = f"{first_octet}.{second_octet}.{third_octet}.{fourth_octet}"

  return ip_address

print(generate_class_c_ip())
```

```
217.126.87.104
```

Figure 5: Python script generating a random Class C IPv4 address — output: 217.126.87.104

A Python script — designed to generate a random Class C IPv4 address — was executed, producing the address 217.126.87.104. A subnet mask of 255.255.255.0 was applied per the lab instructions. The address was converted to binary and the network/host addresses were derived:

Binary conversion of 217.126.87.104:

217 → 11011001
126 → 01111110
87 → 01010111
104 → 01101000

217.126.87.104 → 11011001.01111110.01010111.01101000

Network Address (Standard AND):

IP Address	11011001.01111110.01010111.01101000 (217.126.87.104)
Subnet Mask	11111111.11111111.11111111.00000000 (255.255.255.0)
Network Addr	**11011001.01111110.01010111.00000000 → 217.126.87.0**

Host Address (Inverted Subnet Mask AND):

IP Address	11011001.01111110.01010111.01101000 (217.126.87.104)
Inv. Subnet	00000000.11111111.11111111.11111111 (0.255.255.255)

Host Addr	00000000.01111110.01010111.01101000 → 0.126.87.104

CONCLUSION

This lab provided a structured introduction to network address identification and binary number conversion within the context of real host configuration data. Using the ipconfig command, IPv4, IPv6, subnet mask, default gateway, and MAC address values were retrieved and systematically converted to their binary representations through octet-by-octet decomposition.

The most demanding component of the lab was performing the subnet mask and host mask bitwise AND operations in binary. The volume of ones and zeros involved in these calculations required careful attention to avoid transcription errors, particularly when working across 32-bit address spaces. This process reinforced the importance of methodical, step-by-step binary arithmetic in network address analysis.

The concepts covered in this lab — address classification, binary conversion, and subnet decomposition — form the foundational layer of network administration and routing. A working understanding of how IP addresses are structured and interpreted in binary is essential for configuring, troubleshooting, and securing both home and enterprise network environments.

REFERENCES

R. Mitra, "Lab 01: Identifying Number Formats," The University of Texas at San Antonio, 2025. Accessed: 2/3/2025.

"Panopto," UTSA, 2015. https://utsa.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=6ec5fe69-85a1-4e7c-9d6e-b26c0008ae5e (accessed Dec. 05, 2024).

Generative AI Searches:

N/A

Collaboration:

This lab was completed independently.